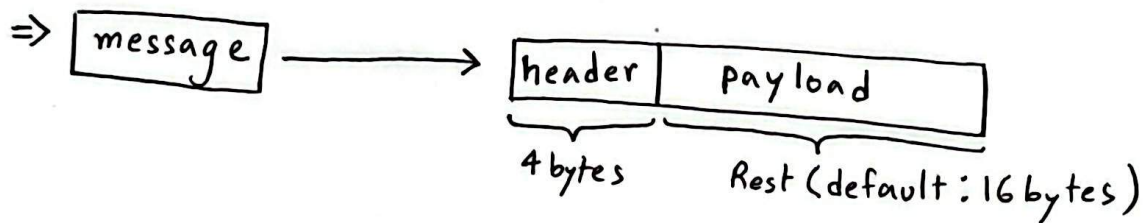
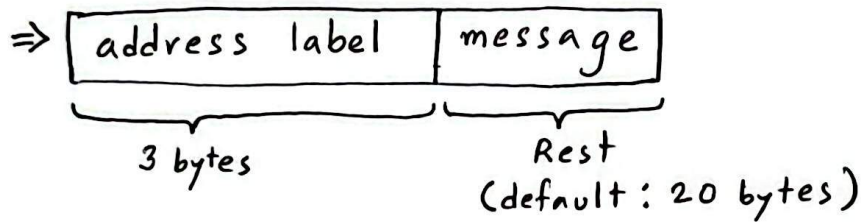
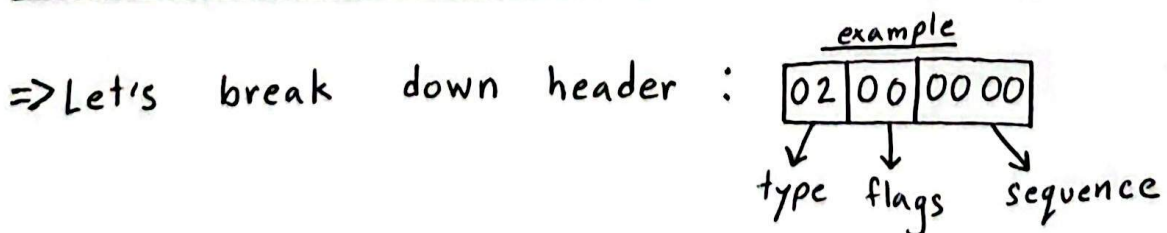


⇒ default packet size (MTU) = 23 bytes



```
abstract final class MessageType {  
    static const int audiochunk = 0x01  
    "           " int control    = 0x02  
    "           " " telemetry  = 0x03  
    "           " " botState   = 0x04  
}
```



type → audio, control, telemetry etc.

flags → extra bits (Mainly for audio)

sequence → packet number. basically used to detect lost packets

```
abstract final class ControlCommandId {  
    static const int setLed = 0x01  
    "           "         " wiggle = 0x02  
    "           "         " playSound = 0x03  
    "           "         " getBattery = 0x04  
}
```

⇒ Let's break down payload: example

01	FF	00	80	01
----	----	----	----	----

• In my code, I'm taking

01

 as setLed

•

FF	00	80
----	----	----

 → setting color

↓ ↓ ↓

RED GREEN BLUE

• Last

01

 is pattern (steady / blinking etc.)

Bottom line: We can define the payload bytes however we want.

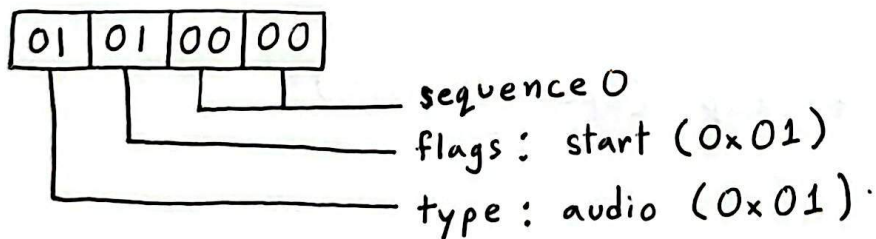
=> Address label :-

	opcode	Read/WRITE etc.
1	Handle (low)	Low byte of characteristic's handle
2	Handle (high)	High byte of characteristic's handle

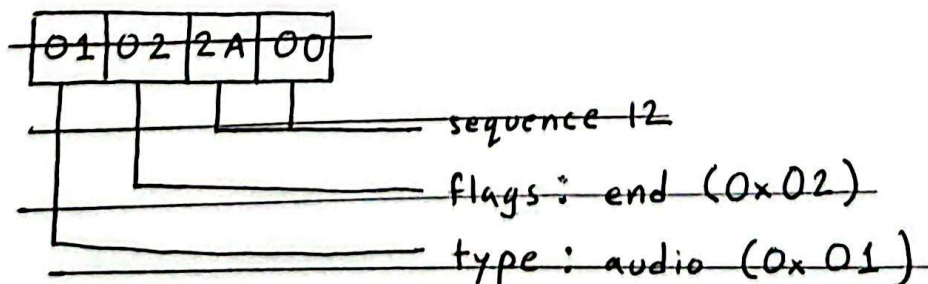
Note: handles are set
by BOT's BLE

4 byte header (AUDIO EXAMPLES)

① First frame of an utterance



② ~~Middle frame (not first, not last)~~



② Middle frame (Not first, Not last)

01	00	05	00
----	----	----	----

sequence 5
flags: none (0x00)
type: audio (0x01)

③ Last frame of an utterance

01	02	2A	00
----	----	----	----

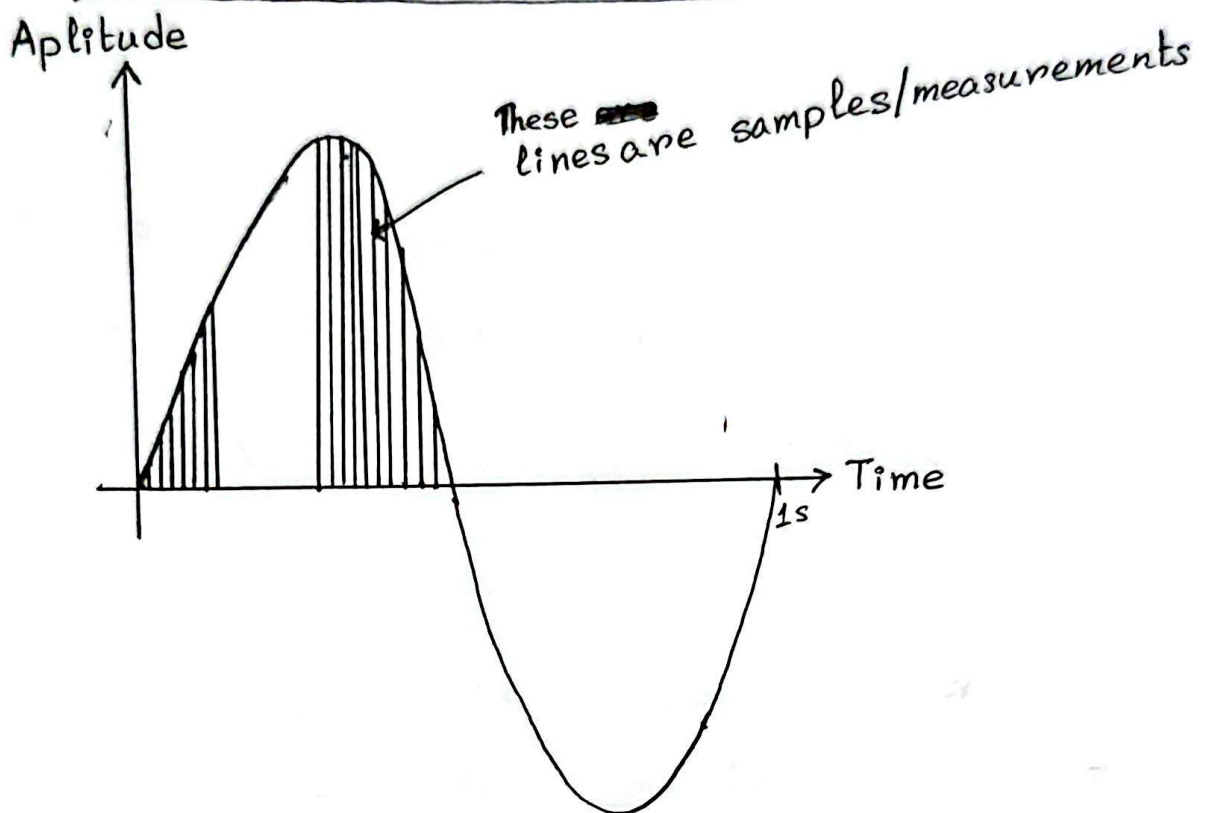
sequence 12
flags: end (0x02)
type: audio (0x01)

④ Very short tap (Special case)

01	03	00	00
----	----	----	----

$0x03 = \text{start}(0x01) + \text{end}(0x02)$

AUDIO CONVERSION PROBLEM



For this project,

$$\begin{aligned} \text{Bitrate} &= 16,000 \text{ samples/sec} \\ &\times 16 \text{ bits/samples} \\ &\times 1 \text{ (channel)} \end{aligned}$$

One mic, One stereo
No left/right stereo

$$\therefore \text{Bitrate} = 256 \text{ kbps}$$

What we need (raw PCM)	What BLE often gives
$\sim 256 \text{ kbps}$	$\sim 50 - 100 \text{ kbps}$

This is the main problem

Solution:- Compress it to ADPCM before sending (adpcm.dart)

Format	Bits per Sample	Compression VS 16 bit pcm
PCM	16 Bits	1X (no compression)
IMA ADPCM	4 Bits	4X smaller

So, bitrate becomes $= (256 \div 4) \text{ kbps} = 64 \text{ kbps}$

Also, we don't send a giant blob of audio, we chunk it. Each chunk is 20 ms of audio.

Why 20 ms?

- Small enough for low latency (walkie-talkie feel)
- Big enough for it to be efficient over BLE
- If one packet is lost, you only lose 20 ms, not the whole sentence.

Each chunk becomes one ADPCM block:-

- 4-byte of ADPCM header (decode state)
- $320 \text{ sample} \times 4 \text{ bits} = 160 \text{ bytes}$ of compressed audio.
- 164 bytes compressed payload.